KR MANGALAM UNIVERSITY

School of Engineering & Technology

B.Tech CSE (AI & Machine Learning) | Batch 2024-2028

PROJECT DOCUMENTATION

SOET Resource Portal

A Secure, Role-Based Academic Resource Management System

Academic Year: 2025-26 | Semester: IV | March 2026

1. Project Overview

The SOET Resource Portal is a full-stack web application developed for KR Mangalam University's School of Engineering & Technology (SOET). It provides a secure, centralised platform where students and faculty can manage and access academic resources — including study materials, previous year question papers (PYQs), subject syllabi, and faculty information — in a structured, role-based environment.

The core problem this portal solves is the fragmented nature of academic resource sharing in universities. Currently, students rely on informal WhatsApp groups, scattered Google Drive links, and word-of-mouth to find study materials. Faculty have no standard way to distribute notes and papers. The SOET Resource Portal replaces this chaos with a single, authenticated, organised system.

Problem Statement

- Students have no single place to find notes and PYQs
- Faculty distribute materials through informal channels
- No access control — anyone can access anything
- No way to verify student identity for resources
- Materials get lost or become inaccessible over time

Our Solution

- Centralised portal with semester-wise subject organisation
- Faculty-only upload system with subject-based authorization
- Role-based access: student, faculty, and admin roles
- Microsoft Azure AD OAuth2 for verified student login
- Permanent MongoDB database with structured file storage

2. Technology Stack

The portal is built using the MERN-adjacent stack — MongoDB, Express.js, Node.js, and EJS — with a Bootstrap 5 frontend. Every technology was chosen deliberately for this specific use case.

Technology	Version / Tool	Why we used it

Node.js	v18+	JavaScript runtime for the server — fast, non-blocking, handles many users simultaneously
Express.js	v4.18	Web framework for routing and middleware — minimal and flexible
MongoDB	v7 (Atlas)	NoSQL database — perfect for storing varied academic data with flexible schemas
Mongoose	v8	ODM for MongoDB — gives us schema validation and easy querying
EJS	v3	Templating engine — renders dynamic HTML pages on the server
Bootstrap 5	v5.3.2	Responsive CSS framework — works on mobile and desktop without extra work
JWT	jsonwebtoken v9	JSON Web Tokens — secure, stateless session management after login
Bcrypt.js	v2.4	Password hashing — stores faculty passwords securely (never plain text)
Multer	v1.4	Handles PDF file uploads — limits type to PDF only, max 20MB
Microsoft Azure AD	OAuth 2.0	Student authentication via official university Outlook accounts
Helmet.js	v7	Adds security headers to every HTTP response — prevents common attacks
express-rate-limit	v7	Limits login attempts to 10 per 15 minutes — prevents brute force
xss-clean + mongo-sanitize	latest	Strips XSS payloads and NoSQL injection from all inputs
Select2	v4.1	Searchable dropdown for subject selection in upload forms
node-fetch	v2	Makes HTTP requests to Microsoft Graph API for student profile retrieval

3. System Architecture

The portal follows a standard MVC (Model-View-Controller) architecture with clear separation of concerns. The backend is a REST-style Express.js server. The frontend is server-rendered using EJS templates. Authentication state is managed through JWT tokens stored in httpOnly cookies.

3.1 Folder Structure

Project Structure

soet-portal/ config/ MongoDB connection setup middleware/ auth.js (JWT verify) + roleCheck.js (student/faculty/admin) models/ 7 Mongoose schemas (Student, Faculty, Course, Subject, etc.) routes/ auth.js, student.js, faculty.js views/ EJS templates (auth/, student/, faculty/, partials/) public/ CSS (main.css) + JS (main.js) uploads/

Stored PDFs (materials/ and pyqs/ folders) seeders/ seed.js — populates database with dummy data utils/ semesterHelper.js + uploadHelper.js app.js Main entry point .env Environment variables (not committed to GitHub)

3.2 Database Collections

The portal uses 7 MongoDB collections, each modelled with Mongoose schemas:

Collection	Key Fields	Purpose
Students	email, rollNo, courseCode, enrollmentYear, batch, section	Stores all 1,200+ student profiles seeded from dummy data
Faculty	email, password, employeeId, designation, specialization, role	Stores 17 faculty accounts with hashed passwords
Courses	courseCode, name, shortName, duration, totalSemesters	6 B.Tech specializations under CSE department
Subjects	subjectCode, name, courseCode, semester, credits, type, category	70+ subjects mapped to courses and semesters
SubjectFacultyMap	subjectCode, facultyId, courseCode, semester, type	Maps which faculty teaches which subject (theory/lab)
StudyMaterials	subjectCode, facultyId, unit, title, fileUrl, uploadedAt	Stores metadata for uploaded study material PDFs
PYQs	subjectCode, year, examType, semesterType, fileUrl, uploadedAt	Stores previous year question papers with exam metadata

3.3 Authentication Flow

The system has two completely separate login paths — one for students and one for faculty/admin:

Student Login Flow (Azure AD OAuth2)

1. Student clicks 'Login as Student (Outlook)' on landing page 2. Browser redirects to Microsoft login (login.microsoftonline.com) 3. Student signs in with their @krmu.edu.in university email 4. Microsoft sends an authorization code back to our callback URL 5. Server exchanges code for access token with Microsoft 6. Server calls Microsoft Graph API to get student email 7. Domain check: email MUST end with @krmu.edu.in (others rejected) 8. Database check: email must exist in our Students collection 9. JWT token created with student data, stored in httpOnly cookie 10. Student redirected to their personalised dashboard

Faculty/Admin Login Flow (Manual)

1. Faculty clicks 'Login as Faculty / Admin' on landing page 2. Enters university email and password on manual login form 3. Rate limiter allows max 10 attempts per 15 minutes (brute force protection) 4. Server finds faculty by email in Faculty collection 5. bcrypt.compare() verifies password against stored hash 6. JWT token created with role ('faculty' or 'admin'), stored in httpOnly cookie 7. Faculty redirected to faculty dashboard

4. Features — Detailed Breakdown

4.1 Student Features

Student Dashboard

After login, the student dashboard displays a complete personalised academic view:

- Profile card showing name, roll number, course, batch, section, and email
- Current semester is auto-calculated from enrollment year and current date using university semester logic (Jan-Jun = even semester, Jul-Nov = odd semester). A student enrolled in 2024 is automatically shown Semester 4 in February 2026.
- Degree progress bar showing percentage of degree completed (Semester 4 of 8 = 50%)
- Four quick-stat cards: number of subjects, total study materials, total PYQs, total credits
- Attendance placeholder card (ready for ERP integration)
- Exam schedule placeholder showing Mid-Term and End-Term slots

Subject Cards

Each enrolled subject is displayed as a card showing:

- Subject code, full name, and short name
- Subject type badge (Theory / Lab / Minor Project)
- Credits with a visual progress bar
- Faculty name and designation for theory and lab separately
- Count of available study materials and PYQs
- Direct action buttons: 'Materials' and 'PYQs' (PYQ button disabled if none uploaded)
- Theory subjects and Lab subjects are shown in separate sections

Study Materials Page

Clicking 'Materials' on any subject card opens a detailed page showing:

- All materials organised unit-by-unit (Unit 1 through Unit 6)
- Each material shows title, description, faculty who uploaded it, and exact upload date-time
- One-click PDF download for each material
- Empty units display a placeholder message rather than hiding

PYQ Page

Clicking 'PYQs' opens a filtered view of previous year papers:

- Filter by year (2020-2026), semester type (odd/even), and exam type (Mid-Term/End-Term)
- Each PYQ card shows year, exam type badge, semester type, uploader, and date
- Direct PDF download for each paper
- Clear filter option to reset to all papers

4.2 Faculty Features

Faculty Dashboard

Faculty see a personalised dashboard with:

- Profile card showing name, employee ID, designation, department, specialization tags, and email
- Four stat cards: assigned subjects, materials uploaded, PYQs uploaded, total uploads

- Quick action buttons for uploading material and PYQs directly
- List of assigned subjects with per-subject material and PYQ counts and shortcut upload links
- Recent uploads section showing last 5 materials and last 5 PYQs, each clickable to open the PDF
- Upload time shown as '12 Jan 2026 at 03:45 PM' format
- Delete button on recent uploads (faculty can only delete their own; admin can delete any)

Upload Study Material

The material upload form includes:

- Searchable subject dropdown (Select2) — faculty type to filter their assigned subjects
- Unit picker: visual grid of Unit 1 through Unit 6, click to select
- Title field with 150 character limit and live counter
- Optional description field with 300 character limit
- Drag-and-drop PDF upload zone — drag a file or click to browse
- Real-time file validation: PDF format check, 20MB size limit, immediate feedback
- Loading spinner shown during upload to prevent double-submission
- Authorization: faculty can only upload for subjects they are mapped to; violation shows error

Upload PYQ

The PYQ upload form includes:

- Searchable subject dropdown (same Select2 as material upload)
- Exam type selection: visual cards for 'Mid-Term' and 'End-Term'
- Year dropdown (last 6 years) and semester type (odd/even)
- Same drag-and-drop PDF zone with validation
- Duplicate prevention: if same subject + year + semester + exam type already exists, upload is blocked

4.3 Admin Features

Admin accounts (set during seeding) have all faculty powers plus:

- ADMIN badge in navbar and dashboard
- Subject list shows ALL subjects across all courses and all faculty
- Recent uploads show ALL uploads from ALL faculty (with uploader name shown)
- Admin Panel (/faculty/admin/all-uploads): table of every material and PYQ in the system
- Admin Panel filter by subject code
- Admin can download and delete any upload regardless of who uploaded it
- Total Students stat card showing all active students in the system
- Password change: admin can change own password through secure form with strength checker

4.4 Security Features

The following security measures are implemented at the application level:

Security Implementation Summary

Rate Limiting: Max 10 login attempts per 15 minutes per IP (express-rate-limit) Password Security: bcrypt with salt rounds 10 for storage; 12 for new passwords JWT Security: Stored in httpOnly cookie (JS cannot read it) with 7-day expiry XSS Protection: xss-clean sanitizes all

request body inputs NoSQL Injection: express-mongo-sanitize strips $ operators from inputs Security Headers: Helmet.js sets CSP, HSTS, X-Frame-Options, etc. Secure Logout: Cookie cleared + no-cache headers prevent back-button access File Validation: PDF MIME type check + extension check + 20MB size limit Authorization: Faculty can only upload for their mapped subjects Domain Check: Only @krmu.edu.in emails allowed for student login

5. Database — Dummy Data

Since official university database access was not available, we built a comprehensive seeder that generates realistic data simulating the actual SOET student and faculty population.

Seeded Data Summary

- 6 B.Tech specializations under CSE
- 1,200+ students across 2023 and 2024 batches
- Realistic names (60 male + 50 female first names)
- 50 common Indian surnames
- Roll numbers follow real format: YY + CourseCode + SeqNo
- University emails: firstname.lastname####@krmu.edu.in
- Students distributed across sections A, B, C
- Your roll 2401730232 is seeded as a real CSE-AIML student

Course Breakdown

- CSE Core (0171): 180+180 students
- CSE-AIML (0173): 120+120 students
- CSE-Data Science (0175): 90+90 students
- CSE-Full Stack (0177): 90+90 students
- CSE-Cloud Computing (0179): 60+60 students
- CSE-UI/UX (0181): 60+60 students
- 17 faculty (Professors to Lab Instructors)
- 200+ subject-faculty mappings

The subjects follow the actual university curriculum structure. Semesters 1 and 2 are common to all branches (Engineering Mathematics, Data Structures, OOP, etc.). From Semester 3, subjects are branch-specific. The AI/ML branch (your branch) has subjects like Machine Learning Fundamentals, Mathematics for ML, Data Visualization, Computer Networks, and Software Engineering in Semester 4 — matching the real curriculum.

6. How We Built It — Phase by Phase

The project was built in 6 structured phases, each with a specific focus area:

Phase	Name	What was built
Phase 1	Project Setup	Node.js + Express initialisation, folder structure (MVC), MongoDB connection, environment configuration, placeholder routes
Phase 2	Database & Models	7 Mongoose schemas, comprehensive seeder with 1,200+ students, 17 faculty, 70+ subjects, 200+ mappings, realistic roll number generation
Phase 3	Authentication	Landing page, Azure AD OAuth2 for students, manual login for faculty, JWT cookie system, role middleware, secure logout with cache-control headers

Phase 4	**Student Dashboard**	Dynamic semester calculation, personalised subject cards, theory/lab separation, study materials page (unit-wise), PYQ page with filters
Phase 5	**Faculty Dashboard**	Upload system for materials and PYQs, drag-drop zone, subject authorization, admin panel, delete functionality, recent uploads with timestamps
Phase 6	**Security & Polish**	Rate limiting, Helmet.js headers, XSS/injection protection, toast notifications, loading spinners, session timeout warning, Select2 search, password change, 404/error pages

7. Team Roles & Contribution

The project is divided into distinct technical areas. Each team member owns one area end-to-end — from building it to being able to explain and demo it to mentors.

Team Member	Role Title	Responsibilities
Member 1	**Backend Lead & Database Architect**	• Designed all 7 Mongoose schemas and their relationships • Built the comprehensive database seeder (1,200+ students, 17 faculty, 70+ subjects) • Implemented subject-faculty mapping system • Set up MongoDB Atlas for cloud database • Designed roll number format and course code system • Explain: How does the database store all the data? Why MongoDB over SQL?
Member 2	**Authentication & Security Engineer**	• Implemented Microsoft Azure AD OAuth2 for student login • Built JWT-based session management (httpOnly cookies) • Implemented role-based middleware (student/faculty/admin) • Set up rate limiting, Helmet.js, XSS and NoSQL injection protection • Implemented secure logout with back-button blocking • Explain: How does student login work? How is it secure?
Member 3	**Student Portal Developer**	• Built the complete student dashboard (profile, semester calc, stats) • Implemented dynamic semester calculation algorithm • Built subject cards with faculty mapping and resource counts • Developed unit-wise study materials page • Developed PYQ page with year/type filters • Explain: How does the portal know which semester a student is in?
Member 4	**Faculty Portal & Upload System**	• Built faculty dashboard with profile and upload statistics

		• Implemented PDF upload system with Multer (material + PYQ) • Built drag-and-drop upload UI with real-time file validation • Implemented authorization check (faculty uploads only for mapped subjects) • Built admin panel for viewing and managing all uploads • Explain: How does the file upload work? How is authorization enforced?
Member 5	**Frontend & UX Designer**	• Designed all EJS view templates with Bootstrap 5 • Built responsive layouts for desktop and mobile • Implemented toast notification system and loading spinners • Created Select2 searchable subject dropdowns • Built professional 404 and error pages • Explain: What makes the UI responsive? What is the design language used?

Note: If your team has fewer than 5 members, combine adjacent roles. For 3 members: Member 1 takes Database + Authentication, Member 2 takes Student Portal + Faculty Portal, Member 3 takes Frontend + Security.

8. How to Demo the Project

8.1 Starting the Application

1. Open the project folder in VS Code
2. Open the integrated terminal (Ctrl + `)
3. Start MongoDB if not already running: mongod
4. Run: npm run dev
5. Open browser: http://localhost:3000

8.2 Demo Script for Mentors

Student Demo (3-4 minutes)

1. Show landing page — explain two login paths 2. Click 'Login as Student (Outlook)' — show Microsoft login redirect (Login with the seeded email: 2401730232@krmu.edu.in if Azure is configured) OR demonstrate the domain check by trying a Gmail account 3. Show student dashboard — explain semester auto-calculation Point out: 'Enrolled 2024, today is Feb 2026, so automatically Semester 4' 4. Click on 'Machine Learning Fundamentals' subject card 5. Show study materials page — unit-wise organisation 6. Go back, click 'PYQs' — show filtering options 7. Demonstrate subject search bar on dashboard 8. Click Logout — then press Back button — show it goes to login, not dashboard

Faculty Demo (3-4 minutes)

1. Login as faculty: dr.rajesh.sharma@krmu.edu.in / Faculty@123 2. Show faculty dashboard — ADMIN badge (this account is admin) 3. Click 'Upload Study Material' - Show searchable dropdown (type 'ML' to filter) - Click Unit 4 - Enter title: 'Unit 4 Notes — Decision Trees' - Drag a PDF onto the drop zone - Show real-time validation, then submit - Show success toast

notification 4. Go back to dashboard — show updated count 5. Click 'Admin Panel' — show all uploads across all subjects 6. Show delete functionality with confirmation 7. Navigate to Change Password — show strength checker

8.3 Key Technical Points to Highlight

- Semester calculation: Enrollment year 2024, current date Feb 2026 = (2026-2024) * 2 = 4th semester. This is automatic and correct for all 1,200 students.
- The JWT token is stored in an httpOnly cookie — this means JavaScript cannot access it, making it immune to XSS token theft.
- Azure AD integration means we never handle student passwords — Microsoft handles all student authentication.
- Faculty authorization: when faculty uploads, the system checks SubjectFacultyMap collection. If their employee ID is not mapped to that subject, the upload is rejected.
- The seeder generates realistic roll numbers: 2401730232 = 24 (year 2024) + 0173 (CSE-AIML course code) + 0232 (roll number 232).

9. Future Roadmap

The following features are planned for future development phases, prioritised by impact:

Phase A — Immediate Next (High Impact)

Notification system — email + in-app alerts for new uploads
Announcement board — faculty post course-specific notices
Syllabus viewer — PDF.js browser-based viewer per subject
Timetable management — weekly schedule per section
AI study assistant (Gemini API — free tier) per subject

Phase B — Medium Term

Assignment submission system with deadlines and grading
Discussion forum — per-subject Q&A with faculty moderation
Bookmarks — students save materials and PYQs
AI mock test generator from uploaded notes and PYQs
Faculty analytics — download counts and engagement data

AI Integration Plan

For AI features, we plan to use Google Gemini 1.5 Flash API (completely free tier: 15 requests/min, 1 million tokens/day, no credit card). Gemini supports native PDF reading which is ideal for our use case — students can ask questions about uploaded study materials and Gemini reads the actual PDF to answer. For real-time chatbot responses, we will use Groq API (also free) which responds in under 1 second using Llama 3.1.

10. Deployment Plan

The portal is currently running locally. The deployment plan uses entirely free-tier cloud services:

Component	Service	Details

Application Hosting	Railway.app	Free $5/month credit — deploys automatically from GitHub on every push
Database	MongoDB Atlas	Free M0 cluster (512MB) on AWS Mumbai — sufficient for our data
File Storage	Local disk (Railway)	Uploaded PDFs stored on Railway filesystem — future upgrade to Cloudinary
Authentication	Microsoft Azure AD	Free OAuth2 app registration — redirect URI updated for production URL
Domain	Railway subdomain	e.g. soet-portal.railway.app — custom domain can be added later
CI/CD	GitHub + Railway	Push to GitHub main branch → Railway auto-deploys in 2 minutes

11. Conclusion

The SOET Resource Portal successfully addresses the core problem of fragmented academic resource management in KR Mangalam University's School of Engineering & Technology. It provides a production-quality, secure, role-based system with:

- Verified student authentication via official Microsoft Outlook accounts
- Structured, semester-wise academic resource organisation for 1,200+ students
- Faculty-controlled upload system with strict authorization enforcement
- Admin oversight with full visibility and management of all content
- Enterprise-grade security (JWT, rate limiting, XSS protection, input sanitization)
- Modern, responsive UI that works on all devices
- A clear, ambitious roadmap including AI-powered features at no cost

The project demonstrates practical application of full-stack web development, database design, OAuth 2.0 authentication, RESTful API design, and information security — all core Computer Science & Engineering competencies. The architecture is production-ready and the codebase is clean, well-structured, and extensible for future feature development.

Project Repository & Access

GitHub Repository: github.com/[YourUsername]/soet-portal (Private) Live Demo URL: soet-portal.railway.app (after deployment) Admin Login: dr.rajesh.sharma@krmu.edu.in / Faculty@123 Faculty Login: ms.ananya.mishra@krmu.edu.in / Faculty@123 Your Student Roll: 2401730232 (use Microsoft Outlook login for students)